

Exercise 1: Control Structures

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    Age INT,  
    Balance DECIMAL(10,2),  
    IsVIP BOOLEAN DEFAULT FALSE  
);
```

```
CREATE TABLE Loans (  
    LoanID INT PRIMARY KEY,  
    CustomerID INT,  
    InterestRate DECIMAL(4,2),  
    DueDate DATE,  
    FOREIGN KEY(CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
INSERT INTO Customers VALUES  
(101, 'Meghana', 65, 15000, FALSE),  
(102, 'Rahul', 45, 8000, FALSE),  
(103, 'Anjali', 72, 22000, FALSE),  
(104, 'Kiran', 61, 9500, FALSE),  
(105, 'Priya', 35, 18000, FALSE);
```

```
INSERT INTO Loans VALUES  
(1001, 101, 8.50, CURDATE()+INTERVAL 15 DAY),  
(1002, 102, 9.00, CURDATE()+INTERVAL 50 DAY),  
(1003, 103, 7.50, CURDATE()+INTERVAL 5 DAY),  
(1004, 104, 8.75, CURDATE()+INTERVAL 28 DAY),
```

(1005,105,9.25,CURDATE()+INTERVAL 90 DAY);

SELECT * FROM Customers;

Result Grid			Filter Rows:	Edit: 	
	CustomerID	CustomerName	Age	Balance	IsVIP
▶	101	Meghana	65	15000.00	0
	102	Rahul	45	8000.00	0
	103	Anjali	72	22000.00	0
	104	Kiran	61	9500.00	0
	105	Priya	35	18000.00	0
*	NULL	NULL	NULL	NULL	NULL

SELECT * FROM Loans;

Result Grid	Filter Rows:	Edit:	Export/Imp
LoanID	CustomerID	InterestRate	DueDate
1001	101	8.50	2026-07-12
1002	102	9.00	2026-08-16
1003	103	7.50	2026-07-02
1004	104	8.75	2026-07-25
1005	105	9.25	2026-09-25
NULL	NULL	NULL	NULL

Scenario 1: Apply 1% Discount to Customers Above 60

DELIMITER \$\$

CREATE PROCEDURE ApplySeniorDiscount()

BEGIN

UPDATE Loans l

JOIN Customers c

ON l.CustomerID=c.CustomerID

SET l.InterestRate=l.InterestRate-1

WHERE c.Age>60;

END\$\$

DELIMITER ;

CALL ApplySeniorDiscount();

```

SELECT
CustomerName,
Age,
InterestRate
FROM Customers c
JOIN Loans l
ON c.CustomerID=l.CustomerID;

```

Result Grid   Filter Rows:			
	CustomerName	Age	InterestRate
▶	Meghana	65	7.50
	Rahul	45	9.00
	Anjali	72	6.50
	Kiran	61	7.75
	Priya	35	9.25

Scenario 2: Promote VIP Customers

DELIMITER \$\$

```
CREATE PROCEDURE PromoteVIP()
```

```
BEGIN
```

```
    UPDATE Customers
```

```
    SET IsVIP=TRUE
```

```
    WHERE Balance>10000;
```

```
END$$
```

DELIMITER ;

```
CALL PromoteVIP();
```

```
SELECT CustomerName,
```

```
Balance,
```

IsVIP

FROM Customers;

Result Grid  Filter Rows: 			
	CustomerName	Balance	IsVIP
▶	Meghana	15000.00	1
	Rahul	8000.00	0
	Anjali	22000.00	1
	Kiran	9500.00	0
	Priya	18000.00	1

Scenario 3: Loan Reminder

DELIMITER \$\$

```
CREATE PROCEDURE LoanReminder()
BEGIN
SELECT
c.CustomerName,
l.LoanID,
l.DueDate,
CONCAT('Reminder: Dear ',c.CustomerName,
', your loan ',l.LoanID,
' is due on ',
DATE_FORMAT(l.DueDate,'%d-%m-%Y')) AS Reminder
FROM Customers c
JOIN Loans l
ON c.CustomerID=l.CustomerID
WHERE l.DueDate BETWEEN CURDATE() AND CURDATE()+INTERVAL 30 DAY;
END$$
```

DELIMITER ;

CALL LoanReminder();

Result Grid				
Filter Rows:		Export:		Wrap Cell Content:
	CustomerName	LoanID	DueDate	Reminder
▶	Meghana	1001	2026-07-12	Reminder: Dear Meghana, your loan 1001 is du...
	Anjali	1003	2026-07-02	Reminder: Dear Anjali, your loan 1003 is due on...
	Kiran	1004	2026-07-25	Reminder: Dear Kiran, your loan 1004 is due on ...

Exercise 2: Error Handling

```
CREATE TABLE Accounts(
    AccountID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Balance DECIMAL(10,2)
);
INSERT INTO Accounts VALUES
(101,'Meghana',15000),
(102,'Rahul',8000),
(103,'Anjali',5000);
```

```
CREATE TABLE Employees(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    Salary DECIMAL(10,2)
);
INSERT INTO Employees VALUES
(1,'Ramesh',50000),
(2,'Suresh',60000),
(3,'Mahesh',70000);
```

```
CREATE TABLE ErrorLog(  
    LogID INT AUTO_INCREMENT PRIMARY KEY,  
    ErrorMessage VARCHAR(255),  
    LogTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Scenario 1: SafeTransferFunds

```
DROP PROCEDURE IF EXISTS SafeTransferFunds;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE SafeTransferFunds(  
    IN FromAcc INT,  
    IN ToAcc INT,  
    IN Amount DECIMAL(10,2)  
)  
BEGIN
```

```
    DECLARE SenderBalance DECIMAL(10,2);
```

```
    START TRANSACTION;
```

```
    SELECT Balance  
    INTO SenderBalance  
    FROM Accounts  
    WHERE AccountID=FromAcc;
```

```
    IF SenderBalance<Amount THEN
```

```
        INSERT INTO ErrorLog(ErrorMessage)  
        VALUES('Insufficient Funds');
```

ROLLBACK;

ELSE

UPDATE Accounts

SET Balance=Balance-Amount

WHERE AccountID=FromAcc;

UPDATE Accounts

SET Balance=Balance+Amount

WHERE AccountID=ToAcc;

COMMIT;

END IF;

END\$\$

DELIMITER ;

CALL SafeTransferFunds(101,102,3000);

SELECT * FROM Accounts;

Result Grid			
Filter Rows:			
	AccountID	CustomerName	Balance
▶	101	Meghana	12000.00
	102	Rahul	11000.00
	103	Anjali	5000.00
✱	NULL	NULL	NULL

Scenario 2: UpdateSalary

DROP PROCEDURE IF EXISTS UpdateSalary;

DELIMITER \$\$

```
CREATE PROCEDURE UpdateSalary(  
  IN EmpID INT,  
  IN PercentIncrease DECIMAL(5,2)  
)
```

BEGIN

IF EXISTS(SELECT * FROM Employees WHERE EmployeeID=EmpID) THEN

```
UPDATE Employees  
SET Salary=Salary+(Salary*PercentIncrease/100)  
WHERE EmployeeID=EmpID;
```

ELSE

```
INSERT INTO ErrorLog(ErrorMessage)  
VALUES(CONCAT('Employee ID ',EmpID,' does not exist'));
```

END IF;

END\$\$

DELIMITER ;

CALL UpdateSalary(1,10);

SELECT * FROM Employees;

Result Grid  Filter Rows: Edit			
	EmployeeID	EmployeeName	Salary
▶	1	Ramesh	55000.00
	2	Suresh	60000.00
	3	Mahesh	70000.00
*	NULL	NULL	NULL

Scenario 3: AddNewCustomer

DROP PROCEDURE IF EXISTS AddNewCustomer;

DELIMITER \$\$

CREATE PROCEDURE AddNewCustomer(

IN CustID INT,

IN CustName VARCHAR(50),

IN CustAge INT,

IN CustBalance DECIMAL(10,2)

)

BEGIN

IF EXISTS(SELECT * FROM Customers WHERE CustomerID=CustID) THEN

INSERT INTO ErrorLog(ErrorMessage)

VALUES(CONCAT('Customer ID ',CustID,' already exists'));

ELSE

INSERT INTO Customers

VALUES(CustID,CustName,CustAge,CustBalance,FALSE);

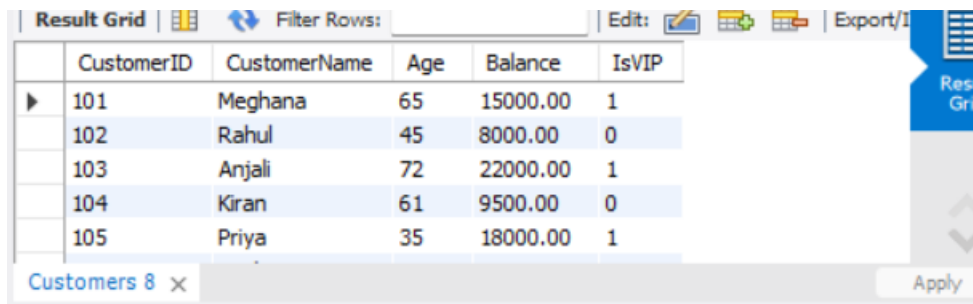
END IF;

END\$\$

DELIMITER ;

CALL AddNewCustomer(106,'Sneha',30,9000);

SELECT * FROM Customers;



	CustomerID	CustomerName	Age	Balance	IsVIP
▶	101	Meghana	65	15000.00	1
	102	Rahul	45	8000.00	0
	103	Anjali	72	22000.00	1
	104	Kiran	61	9500.00	0
	105	Priya	35	18000.00	1

Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest

```
CREATE TABLE SavingsAccounts(  
    AccountID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    Balance DECIMAL(10,2)  
);
```

```
INSERT INTO SavingsAccounts VALUES  
(201,'Meghana',10000),  
(202,'Rahul',25000),
```

```
(203,'Anjali',50000),  
(204,'Kiran',15000);
```

```
DROP PROCEDURE IF EXISTS ProcessMonthlyInterest;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ProcessMonthlyInterest()
```

```
BEGIN
```

```
UPDATE SavingsAccounts
```

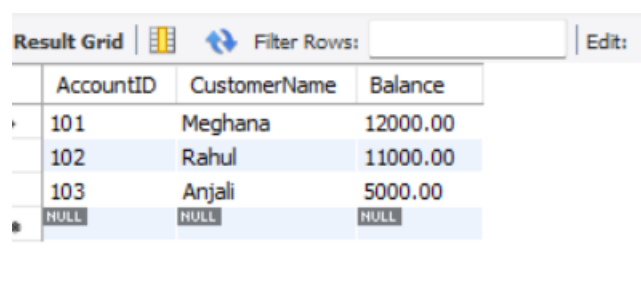
```
SET Balance = Balance + (Balance * 0.01);
```

```
END$$
```

```
DELIMITER ;
```

```
CALL ProcessMonthlyInterest();
```

```
SELECT * FROM SavingsAccounts;
```



The screenshot shows a 'Result Grid' window with a table containing 4 rows and 3 columns: AccountID, CustomerName, and Balance. The first three rows contain data for accounts 101, 102, and 103. The fourth row shows NULL values for all three columns. The Balance values are 12000.00, 11000.00, and 5000.00 respectively. The window also includes a 'Filter Rows' search bar and an 'Edit' button.

AccountID	CustomerName	Balance
101	Meghana	12000.00
102	Rahul	11000.00
103	Anjali	5000.00
NULL	NULL	NULL

Scenario 2: Update Employee Bonus

```
ALTER TABLE Employees
```

```
ADD Department VARCHAR(30);
```

```
UPDATE Employees
```

```
SET Department='IT'  
WHERE EmployeeID=1;
```

```
UPDATE Employees  
SET Department='HR'  
WHERE EmployeeID=2;
```

```
UPDATE Employees  
SET Department='IT'  
WHERE EmployeeID=3;
```

```
DROP PROCEDURE IF EXISTS UpdateEmployeeBonus;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE UpdateEmployeeBonus(  
IN Dept VARCHAR(30),  
IN BonusPercent DECIMAL(5,2)  
)
```

```
BEGIN
```

```
UPDATE Employees  
SET Salary = Salary + (Salary * BonusPercent / 100)  
WHERE Department = Dept;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL UpdateEmployeeBonus('IT',10);
```

```
SELECT * FROM Employees;
```

Result Grid

Filter Rows:

Edit:

	EmployeeID	EmployeeName	Salary	Department
▶	1	Ramesh	60500.00	IT
	2	Suresh	60000.00	HR
	3	Maresh	77000.00	IT
⬤	NULL	NULL	NULL	NULL

Scenario 3: Transfer Funds

```
DROP PROCEDURE IF EXISTS TransferFunds;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE TransferFunds(
```

```
IN FromAccount INT,
```

```
IN ToAccount INT,
```

```
IN Amount DECIMAL(10,2)
```

```
)
```

```
BEGIN
```

```
DECLARE BalanceAvailable DECIMAL(10,2);
```

```
SELECT Balance
```

```
INTO BalanceAvailable
```

```
FROM Accounts
```

```
WHERE AccountID = FromAccount;
```

```
IF BalanceAvailable >= Amount THEN
```

```
UPDATE Accounts
```

```
SET Balance = Balance - Amount
```

```
WHERE AccountID = FromAccount;
```

```
UPDATE Accounts
```

```
SET Balance = Balance + Amount
```

```
WHERE AccountID = ToAccount;
```

```
SELECT 'Transfer Successful' AS Status;
```

```
ELSE
```

```
SELECT 'Insufficient Balance' AS Status;
```

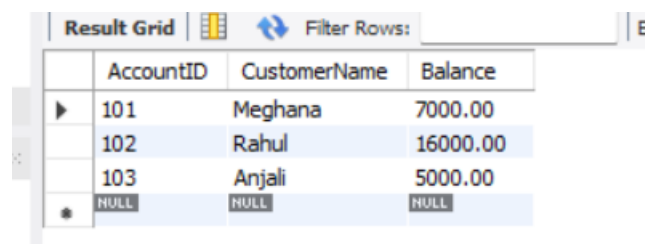
```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL TransferFunds(101,102,5000);
```

```
SELECT * FROM Accounts;
```



The screenshot shows a 'Result Grid' window with a table containing account information. The table has columns for AccountID, CustomerName, and Balance. The data is as follows:

AccountID	CustomerName	Balance
101	Meghana	7000.00
102	Rahul	16000.00
103	Anjali	5000.00
NULL	NULL	NULL

Exercise 4: Functions

Scenario 1: CalculateAge

```
ALTER TABLE Customers
```

```
ADD DateOfBirth DATE;
```

```
UPDATE Customers
```

```
SET DateOfBirth='2002-05-10'
```

```
WHERE CustomerID=101;
```

```
UPDATE Customers
```

```
SET DateOfBirth='1995-08-20'
```

```
WHERE CustomerID=102;
```

```
UPDATE Customers
```

```
SET DateOfBirth='1988-01-15'
```

```
WHERE CustomerID=103;
```

```
UPDATE Customers
```

```
SET DateOfBirth='1964-11-25'
```

```
WHERE CustomerID=104;
```

```
UPDATE Customers
```

```
SET DateOfBirth='1998-03-18'
```

```
WHERE CustomerID=105;
```

```
DROP FUNCTION IF EXISTS CalculateAge;
```

```
DELIMITER $$
```

```

CREATE FUNCTION CalculateAge(DOB DATE)
RETURNS INT
DETERMINISTIC

BEGIN

RETURN TIMESTAMPDIFF(YEAR,DOB,CURDATE());

END$$

DELIMITER ;

SELECT
CustomerName,
DateOfBirth,
CalculateAge(DateOfBirth) AS Age
FROM Customers;

```

Result Grid			
Filter Rows:			
Export:			
	CustomerName	DateOfBirth	Age
▶	Meghana	2002-05-10	24
	Rahul	1995-08-20	30
	Anjali	1988-01-15	38
	Kiran	1964-11-25	61
	Priya	1998-03-18	28

Scenario 2: CalculateMonthlyInstallment

```

DROP FUNCTION IF EXISTS CalculateMonthlyInstallment;

```

```

DELIMITER $$

```



```
CREATE FUNCTION CalculateMonthlyInstallment(  
LoanAmount DECIMAL(10,2),  
InterestRate DECIMAL(5,2),  
Years INT  
)
```

```
RETURNS DECIMAL(10,2)
```

```
DETERMINISTIC
```

```
BEGIN
```

```
DECLARE EMI DECIMAL(10,2);
```

```
SET EMI =
```

```
(LoanAmount +
```

```
(LoanAmount*InterestRate*Years/100))
```

```
/
```

```
(Years*12);
```

```
RETURN EMI;
```

```
END$$
```

```
DELIMITER ;
```

```
SELECT CalculateMonthlyInstallment(500000,8.5,5) AS MonthlyInstallment;
```

	MonthlyInstallment
▶	11875.00

Scenario 3: HasSufficientBalance

DROP FUNCTION IF EXISTS HasSufficientBalance;

DELIMITER \$\$

```
CREATE FUNCTION HasSufficientBalance(  
  AccID INT,  
  Amount DECIMAL(10,2)  
)
```

RETURNS BOOLEAN

READS SQL DATA

BEGIN

DECLARE CurrentBalance DECIMAL(10,2);

SELECT Balance

INTO CurrentBalance

FROM Accounts

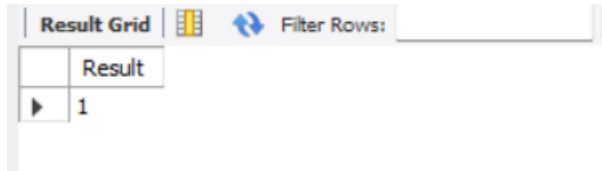
WHERE AccountID=AccID;

RETURN CurrentBalance>=Amount;

END\$\$

DELIMITER ;

SELECT HasSufficientBalance(101,5000) AS Result;



The screenshot shows a 'Result Grid' window. It has a toolbar with icons for grid view, table view, and a 'Filter Rows' input field. Below the toolbar is a table with one column labeled 'Result' and one row containing the value '1'.

Result
1

Exercise 5: Triggers

Scenario 1: Update Customer Last Modified

ALTER TABLE Customers

ADD LastModified DATETIME;

DROP TRIGGER IF EXISTS UpdateCustomerLastModified;

DELIMITER \$\$

CREATE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE

ON Customers

FOR EACH ROW

BEGIN

SET NEW.LastModified = NOW();

END\$\$

DELIMITER ;

UPDATE Customers

SET Balance = 20000

WHERE CustomerID = 101;

SELECT

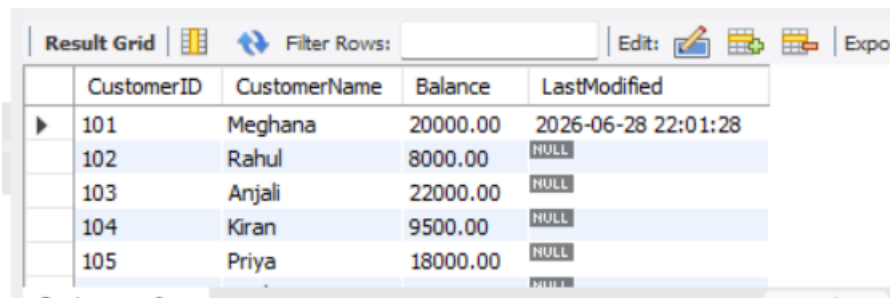
CustomerID,

CustomerName,

Balance,

LastModified

FROM Customers;



The screenshot shows a SQL query result grid with the following data:

CustomerID	CustomerName	Balance	LastModified
101	Meghana	20000.00	2026-06-28 22:01:28
102	Rahul	8000.00	NULL
103	Anjali	22000.00	NULL
104	Kiran	9500.00	NULL
105	Priya	18000.00	NULL

Scenario 2: Maintain Audit Log

CREATE TABLE Transactions(

TransactionID INT PRIMARY KEY,

AccountID INT,

TransactionType VARCHAR(20),

Amount DECIMAL(10,2),

TransactionDate DATETIME DEFAULT NOW())

);

CREATE TABLE AuditLog(

LogID INT AUTO_INCREMENT PRIMARY KEY,

TransactionID INT,

Message VARCHAR(255),

LogTime DATETIME DEFAULT NOW()

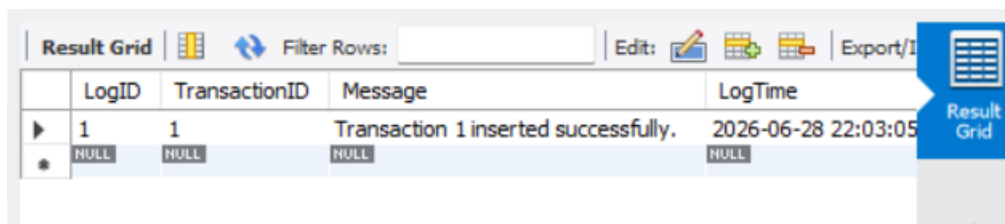
);

INSERT INTO Transactions

VALUES

(1,101,'Deposit',5000,NOW());

SELECT * FROM AuditLog;



The screenshot shows a database query result grid with the following columns: LogID, TransactionID, Message, and LogTime. The first row contains the values 1, 1, Transaction 1 inserted successfully., and 2026-06-28 22:03:05. The second row shows NULL values for all columns. The interface includes a toolbar with options like Filter Rows, Edit, and Export/Import.

	LogID	TransactionID	Message	LogTime
▶	1	1	Transaction 1 inserted successfully.	2026-06-28 22:03:05
✱	NULL	NULL	NULL	NULL

Scenario 3: Check Transaction Rules

DROP TRIGGER IF EXISTS CheckTransactionRules;

DELIMITER \$\$

CREATE TRIGGER CheckTransactionRules

BEFORE INSERT

ON Transactions

FOR EACH ROW

BEGIN

DECLARE CurrentBalance DECIMAL(10,2);

IF NEW.TransactionType='Deposit'

AND NEW.Amount<=0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Deposit amount must be positive';

END IF;

IF NEW.TransactionType='Withdrawal' THEN

SELECT Balance

INTO CurrentBalance

FROM Accounts

WHERE AccountID=NEW.AccountID;

IF NEW.Amount>CurrentBalance THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Insufficient Balance';

END IF;

END IF;

END\$\$

DELIMITER ;

Test 1 (Valid Deposit):

INSERT INTO Transactions

VALUES

(2,101,'Deposit',1000,NOW());

Test 2 (Invalid Deposit):

INSERT INTO Transactions

VALUES

(3,101,'Deposit',-500,NOW());

Test 3 (Withdrawal Within Balance):

INSERT INTO Transactions

VALUES

(4,101,'Withdrawal',3000,NOW());

Test 4 (Withdrawal Exceeding Balance):

INSERT INTO Transactions

VALUES

(5,103,'Withdrawal',50000,NOW());

Exercise 6: Cursors

Scenario 1: Generate Monthly Statements

```
INSERT INTO Transactions VALUES
(101,101,'Deposit',5000,CURDATE()),
(102,102,'Withdrawal',2000,CURDATE()),
(103,103,'Deposit',3000,CURDATE()),
(104,101,'Withdrawal',1000,CURDATE());

DROP PROCEDURE IF EXISTS GenerateMonthlyStatements;

DELIMITER $$

CREATE PROCEDURE GenerateMonthlyStatements()

BEGIN

DECLARE done INT DEFAULT FALSE;

DECLARE vName VARCHAR(50);
DECLARE vType VARCHAR(20);
DECLARE vAmount DECIMAL(10,2);
DECLARE vDate DATE;

DECLARE cur CURSOR FOR

SELECT
c.CustomerName,
t.TransactionType,
t.Amount,
```



```
t.TransactionDate
FROM Customers c
JOIN Transactions t
ON c.CustomerID=t.AccountID
WHERE MONTH(t.TransactionDate)=MONTH(CURDATE())
AND YEAR(t.TransactionDate)=YEAR(CURDATE());

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done=TRUE;

OPEN cur;

read_loop: LOOP

FETCH cur INTO
vName,
vType,
vAmount,
vDate;

IF done THEN
LEAVE read_loop;
END IF;

SELECT CONCAT(
'Customer: ',vName,
' | Type: ',vType,
' | Amount: ',vAmount,
' | Date: ',vDate
) AS MonthlyStatement;
```

```
END LOOP;
```

```
CLOSE cur;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL GenerateMonthlyStatements();
```

Result Grid	Filter Rows:	Export: 	Wrap Cell C
	MonthlyStatement		
▶	Customer: Meghana Type: Withdrawal Amou...		

Scenario 2: Apply Annual Fee

```
DROP PROCEDURE IF EXISTS ApplyAnnualFee;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ApplyAnnualFee()
```

```
BEGIN
```

```
DECLARE done INT DEFAULT FALSE;
```

```
DECLARE vAccount INT;
```

```
DECLARE cur CURSOR FOR
```

SELECT AccountID

FROM Accounts;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done=TRUE;

OPEN cur;

fee_loop: LOOP

FETCH cur INTO vAccount;

IF done THEN

LEAVE fee_loop;

END IF;

UPDATE Accounts

SET Balance=Balance-500

WHERE AccountID=vAccount;

END LOOP;

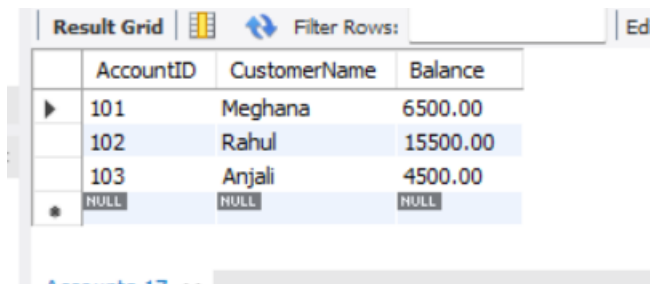
CLOSE cur;

END\$\$

DELIMITER ;

```
CALL ApplyAnnualFee();
```

```
SELECT * FROM Accounts;
```



The screenshot shows a SQL query result grid with the following data:

AccountID	CustomerName	Balance
101	Meghana	6500.00
102	Rahul	15500.00
103	Anjali	4500.00
NULL	NULL	NULL

Scenario 3: Update Loan Interest

```
DROP PROCEDURE IF EXISTS UpdateLoanInterestRates;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE UpdateLoanInterestRates()
```

```
BEGIN
```

```
DECLARE done INT DEFAULT FALSE;
```

```
DECLARE vLoan INT;
```

```
DECLARE vRate DECIMAL(5,2);
```

```
DECLARE cur CURSOR FOR
```

```
SELECT LoanID,InterestRate
```

```
FROM Loans;
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done=TRUE;
```

```
OPEN cur;
```

```
loan_loop: LOOP
```

```
FETCH cur INTO
```

```
vLoan,
```

```
vRate;
```

```
IF done THEN
```

```
LEAVE loan_loop;
```

```
END IF;
```

```
IF vRate>8 THEN
```

```
UPDATE Loans
```

```
SET InterestRate=InterestRate-0.5
```

```
WHERE LoanID=vLoan;
```

```
ELSE
```

```
UPDATE Loans
```

```
SET InterestRate=InterestRate-0.25
```

```
WHERE LoanID=vLoan;
```

END IF;

END LOOP;

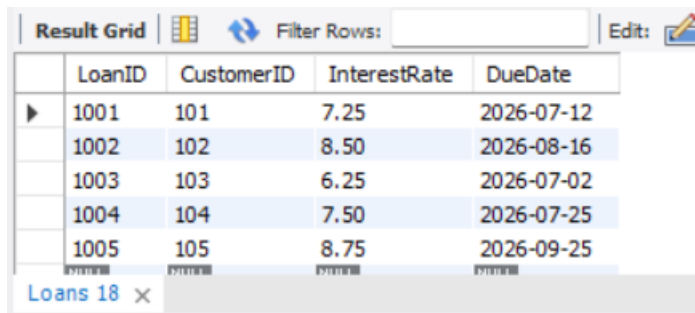
CLOSE cur;

END\$\$

DELIMITER ;

CALL UpdateLoanInterestRates();

SELECT * FROM Loans;



The screenshot shows a database result grid with the following data:

	LoanID	CustomerID	InterestRate	DueDate
▶	1001	101	7.25	2026-07-12
	1002	102	8.50	2026-08-16
	1003	103	6.25	2026-07-02
	1004	104	7.50	2026-07-25
	1005	105	8.75	2026-09-25

Below the table, there is a tab labeled "Loans 18" with a close button (x).

Exercise 7: Packages

Scenario 1 : CustomerManagement Package

DROP PROCEDURE IF EXISTS CustomerManagement_AddCustomer;

DELIMITER \$\$

CREATE PROCEDURE CustomerManagement_AddCustomer(

IN pID INT,

IN pName VARCHAR(50),

IN pAge INT,

IN pBalance DECIMAL(10,2)

)

BEGIN

INSERT INTO Customers

(CustomerID, CustomerName, Age, Balance, IsVIP)

VALUES

(pID, pName, pAge, pBalance, FALSE);

END\$\$

DELIMITER ;

CALL CustomerManagement_AddCustomer

(110, 'Ravi', 30, 20000);

DROP PROCEDURE IF EXISTS CustomerManagement_UpdateCustomer;

DELIMITER \$\$

CREATE PROCEDURE CustomerManagement_UpdateCustomer(

IN pID INT,

IN pBalance DECIMAL(10,2)

)

BEGIN

UPDATE Customers

SET Balance=pBalance

WHERE CustomerID=pID;

END\$\$

DELIMITER ;

CALL CustomerManagement_UpdateCustomer

(110,25000);

DROP FUNCTION IF EXISTS CustomerManagement_GetBalance;

DELIMITER \$\$

CREATE FUNCTION CustomerManagement_GetBalance(

pID INT

)

RETURNS DECIMAL(10,2)

READS SQL DATA

BEGIN


```
DECLARE bal DECIMAL(10,2);
```

```
SELECT Balance
```

```
INTO bal
```

```
FROM Customers
```

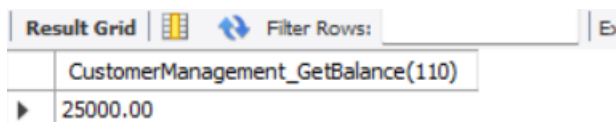
```
WHERE CustomerID=pID;
```

```
RETURN bal;
```

```
END$$
```

```
DELIMITER ;
```

```
SELECT CustomerManagement_GetBalance(110);
```



Result Grid		Filter Rows:	Ex
	CustomerManagement_GetBalance(110)		
▶	25000.00		

Scenario 2 : EmployeeManagement Package

```
DROP PROCEDURE IF EXISTS EmployeeManagement_HireEmployee;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeManagement_HireEmployee(
```

```
IN id INT,
```

```
IN name VARCHAR(50),  
IN salary DECIMAL(10,2),  
IN dept VARCHAR(30)
```

```
)
```

```
BEGIN
```

```
INSERT INTO Employees
```

```
VALUES
```

```
(id,name,salary,dept);
```

```
END$$
```

```
DELIMITER ;
```

```
CALL EmployeeManagement_HireEmployee
```

```
(10,'Kavya',50000,'IT');
```

```
DROP PROCEDURE IF EXISTS EmployeeManagement_UpdateEmployee;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE EmployeeManagement_UpdateEmployee(
```

```
IN id INT,
```

```
IN salary DECIMAL(10,2)
```

)

BEGIN

UPDATE Employees

SET Salary=salary

WHERE EmployeeID=id;

END\$\$

DELIMITER ;

CALL EmployeeManagement_UpdateEmployee

(10,65000);

DROP FUNCTION IF EXISTS EmployeeManagement_AnnualSalary;

DELIMITER \$\$

CREATE FUNCTION EmployeeManagement_AnnualSalary(

id INT

)

RETURNS DECIMAL(10,2)

READS SQL DATA

BEGIN

DECLARE annual DECIMAL(10,2);

SELECT Salary*12

INTO annual

FROM Employees

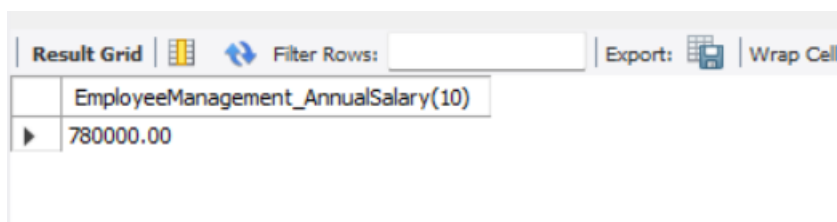
WHERE EmployeeID=id;

RETURN annual;

END\$\$

DELIMITER ;

SELECT EmployeeManagement_AnnualSalary(10);



The screenshot shows a SQL query result grid. The grid has a header row with the column name 'EmployeeManagement_AnnualSalary(10)' and a data row with the value '780000.00'. The grid is displayed in a window with a toolbar at the top containing buttons for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell'.

EmployeeManagement_AnnualSalary(10)
780000.00

Scenario 3 : AccountOperations Package

DROP PROCEDURE IF EXISTS AccountOperations_OpenAccount;

DELIMITER \$\$

CREATE PROCEDURE AccountOperations_OpenAccount(

IN id INT,

IN name VARCHAR(50),

IN balance DECIMAL(10,2)

)

BEGIN

INSERT INTO Accounts

VALUES

(id,name,balance);

END\$\$

DELIMITER ;

CALL AccountOperations_OpenAccount

(200,'Arjun',50000);

DROP PROCEDURE IF EXISTS AccountOperations_CloseAccount;

DELIMITER \$\$

CREATE PROCEDURE AccountOperations_CloseAccount(

IN id INT

)

BEGIN

DELETE FROM Accounts

WHERE AccountID=id;

END\$\$

DELIMITER ;

CALL AccountOperations_CloseAccount(200);

DROP FUNCTION IF EXISTS AccountOperations_TotalBalance;

DELIMITER \$\$

CREATE FUNCTION AccountOperations_TotalBalance(

id INT

)

RETURNS DECIMAL(10,2)

READS SQL DATA

BEGIN

```
DECLARE total DECIMAL(10,2);
```

```
SELECT Balance
```

```
INTO total
```

```
FROM Accounts
```

```
WHERE AccountID=id;
```

```
RETURN total;
```

```
END$$
```

```
DELIMITER ;
```

```
SELECT AccountOperations_TotalBalance(101);
```

Result Grid	Filter Rows:	Export:	Wrap Cell Contents:
AccountOperations_TotalBalance(101)			
▶ 6500.00			